

ISSUES UI IMPLEMENTATION STRATEGY

v0.1.5

Targeted UX Friction Resolution & Advanced Graph Visualisation Architecture

FROM: UI/Frontend Claude Code Session
TO: Backend Team / Review
DATE: 2 February 2026
STATUS: Ready for Review

Strategic Focus: Stabilisation Before Expansion

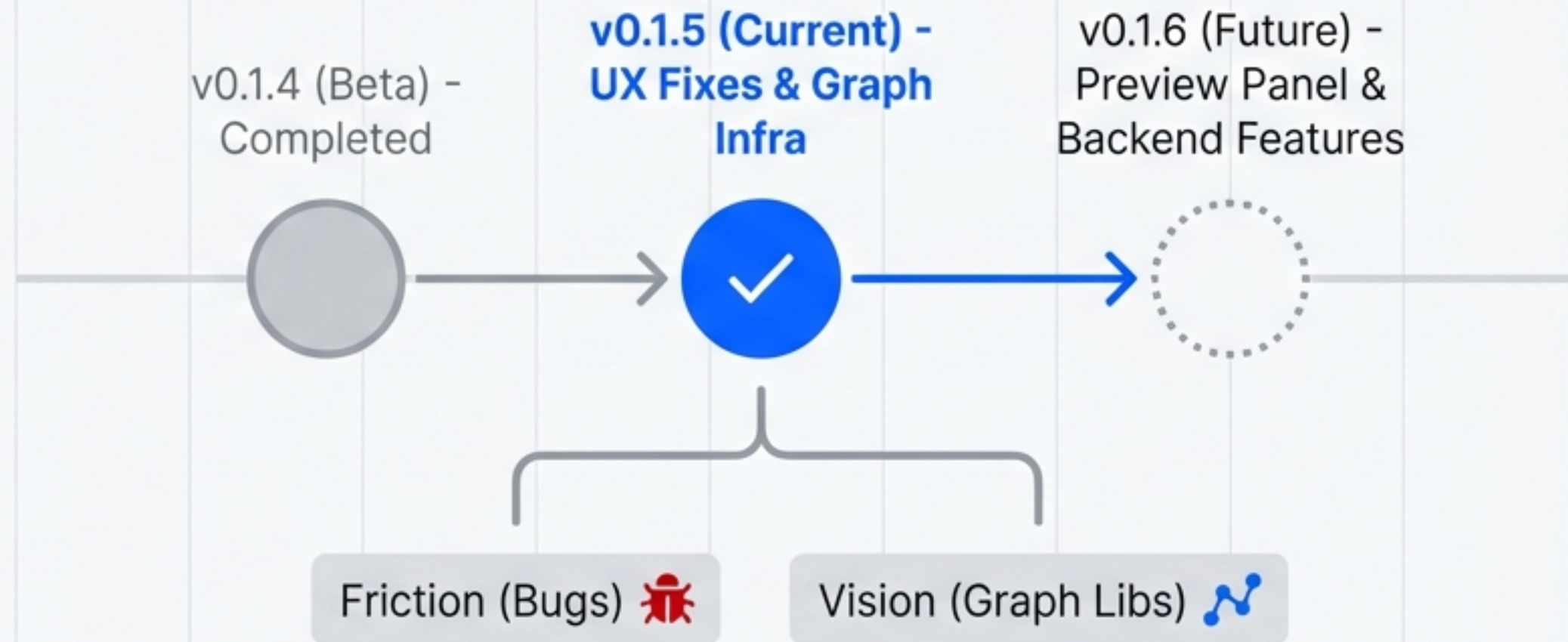
v0.1.5 is a targeted release designed to stabilise user workflows and establish the visualisation infrastructure before we tackle larger backend features.

Key Drivers

- **Source:** Backlog review & v0.1.4 testing friction points.
- **Primary Goal:** UX Improvements & Workflow Enhancements.

The Approach

1. **Smoothing Friction:** Addressing data loss and navigation breaks.
2. **Expanding Vision:** Implementing a multi-library graph viewer.



Scope Definition: In vs. Out

IN SCOPE

- Bug-7: Modal Data Safety **HIGH**
- Bug-8: Back Navigation Logic **HIGH**
- Task-18: New Button in Detail **MEDIUM**
- Task-23: Inline Confirmations **MEDIUM**
- Task-25: Kanban Tab Handling **MEDIUM**
- Tasks 19-21: Graph Visualisation Suite **MEDIUM**

DEFERRED

Backend Dependent: Bug-9, Bug-10, Task-22

Future Scope (v0.1.6): Task-24 (Preview Panel)

Future Scope: Feature-10

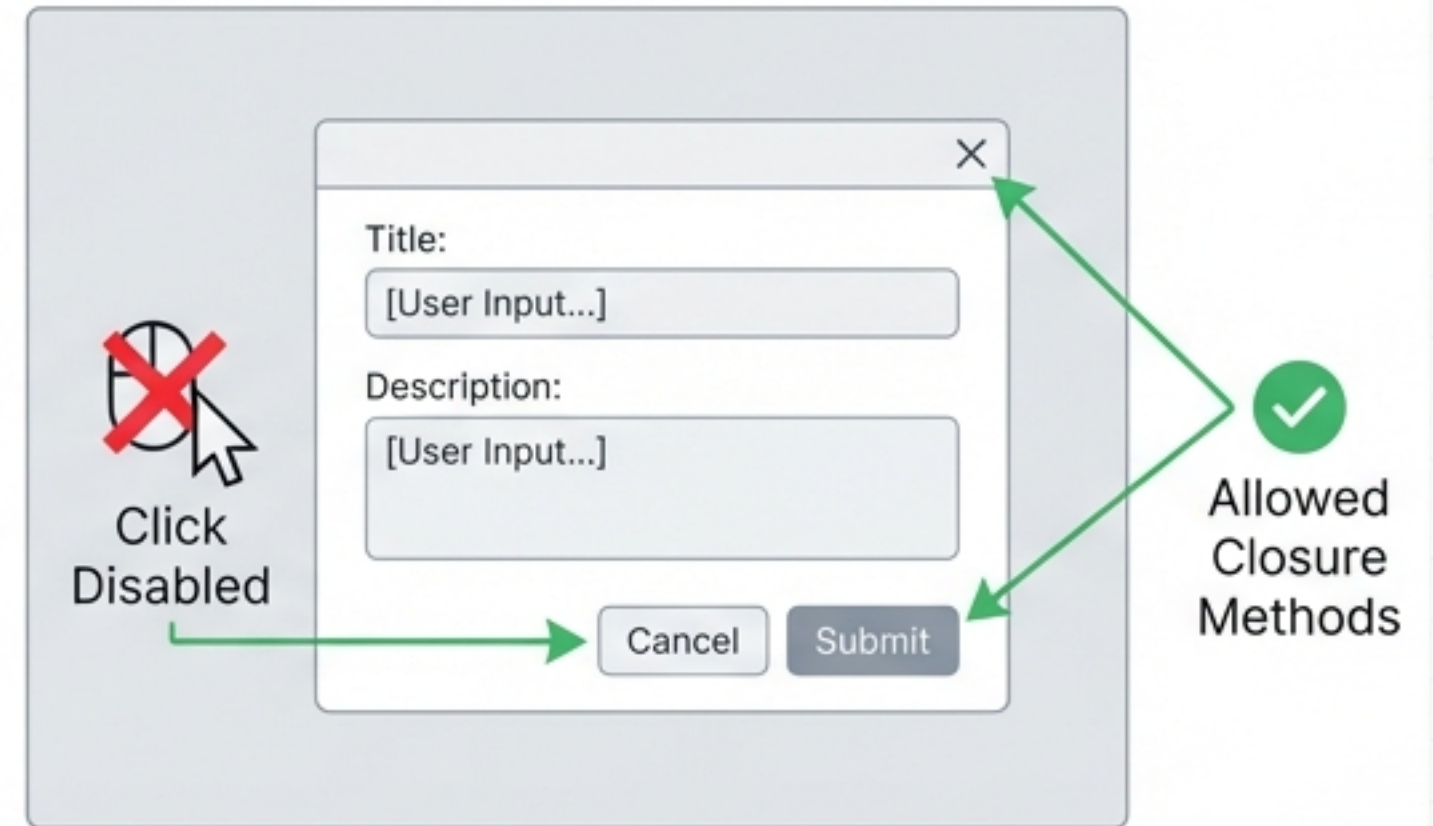
Low Priority: Task-17 (Refactor)

Protecting User Intent: Data Retention in Modals (Bug-7)

The Problem: In v0.1.4, an accidental click outside the modal overlay wipes all user input without warning.

The Solution: Option A (Recommended) - Disable 'click-outside-to-close'.

Rationale: Prevents accidental data loss entirely. User must explicitly click Cancel or 'X'.



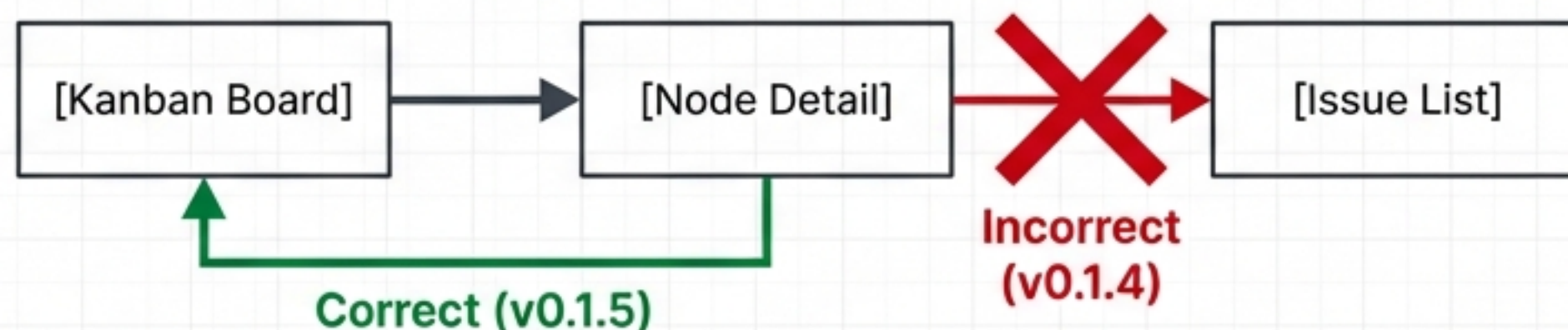
File: `v0.1.5/components/create-node-modal/create-node-modal.js`

Action: Modify overlay click handler to block closure.

Repairing the Mental Model: Navigation & Multi-tasking

Challenge 1: The Broken Back Button (Bug-8)

Issue: Clicking 'Back' from Node Detail incorrectly sends users to the Issue List, ignoring their origin (Kanban/Graph).



Fix: Implement history tracking in 'issues-shell.js'.

Challenge 2: The Kanban Trap (Task-25)

Issue: Users cannot Cmd+Click Kanban cards to open new tabs.

Technical Solution: Wrap Kanban cards in native `<a>` tags.

```
<a href='#/issue/123' onClick={handleSpaNav}>
  <KanbanCard />
</a>
```

Must preserve Drag-and-Drop functionality on 'mousedown'.

Reducing Workflow Friction: Speed & Context

Task-18: Continuous Creation

Problem: Too many clicks to create consecutive issues.

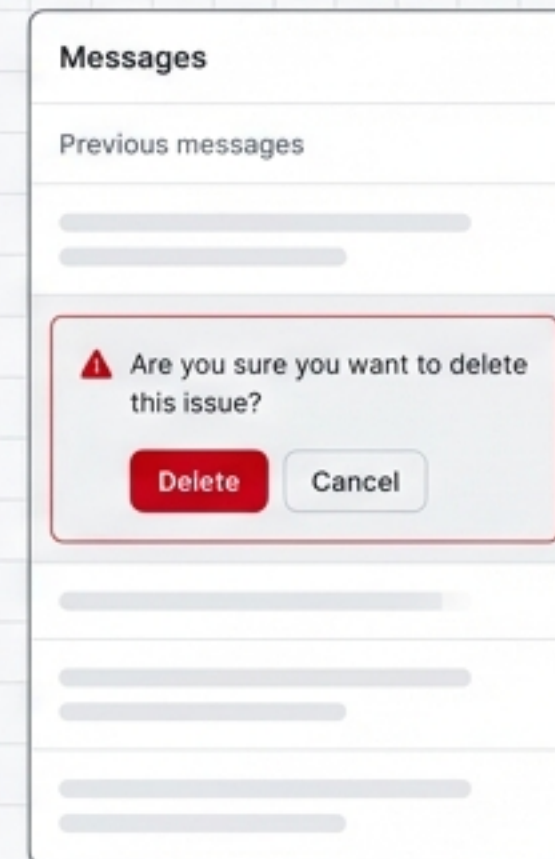
Solution: Add persistent '+ New' button in Node Detail header.



Task-23: Non-Blocking Confirmations

Problem: Native 'confirm()' dialogs are disruptive.

Solution: Inline UI within Messages Panel.



Implementation: Update 'messages-service.js' to support 'confirm' type.

The Visualisation Strategy: A Multi-Mode Approach

Core Philosophy: One graph library cannot satisfy all user needs. We are implementing a specialized suite.

Mermaid.js



The Documenter

Use Case:
Sharing & Specs

Vis.js



The Performer

Use Case: Large
Datasets & Mobile

Cytoscape.js



The Analyst

Use Case:
Network Science

Controlled by central 'graph-viewer.js' Architecture

Mode 1: Mermaid.js (The Documenter)

Target Use Case: Documentation,
GitHub/GitLab integration, and quick sharing.

Key Advantages:

- Declarative Syntax (Text-based)
- Auto-layout
- Easy export to Markdown

Implementation (Task-19):

- Load CDN in 'index.html'
- New Component: 'graph-viewer-mermaid.js'
- Feature: 'Copy Mermaid Code' button

API JSON Data

```
{
  "nodes": [
    { "id": "A", "label": "Task 1" },
    { "id": "B", "label": "Decision" },
    { "id": "C", "label": "Result" }
  ],
  "edges": [
    { "source": "A", "target": "B" },
    { "source": "B", "target": "C", "label": "Yes" }
  ]
}
```



```
mermaid
graph TD
  A[Task 1] --> B{Decision}
  B -->|Yes| C[Result]
```

Copy Mermaid Code

Mermaid Syntax & Rendered View



Mode 2: Vis.js (The Performer)

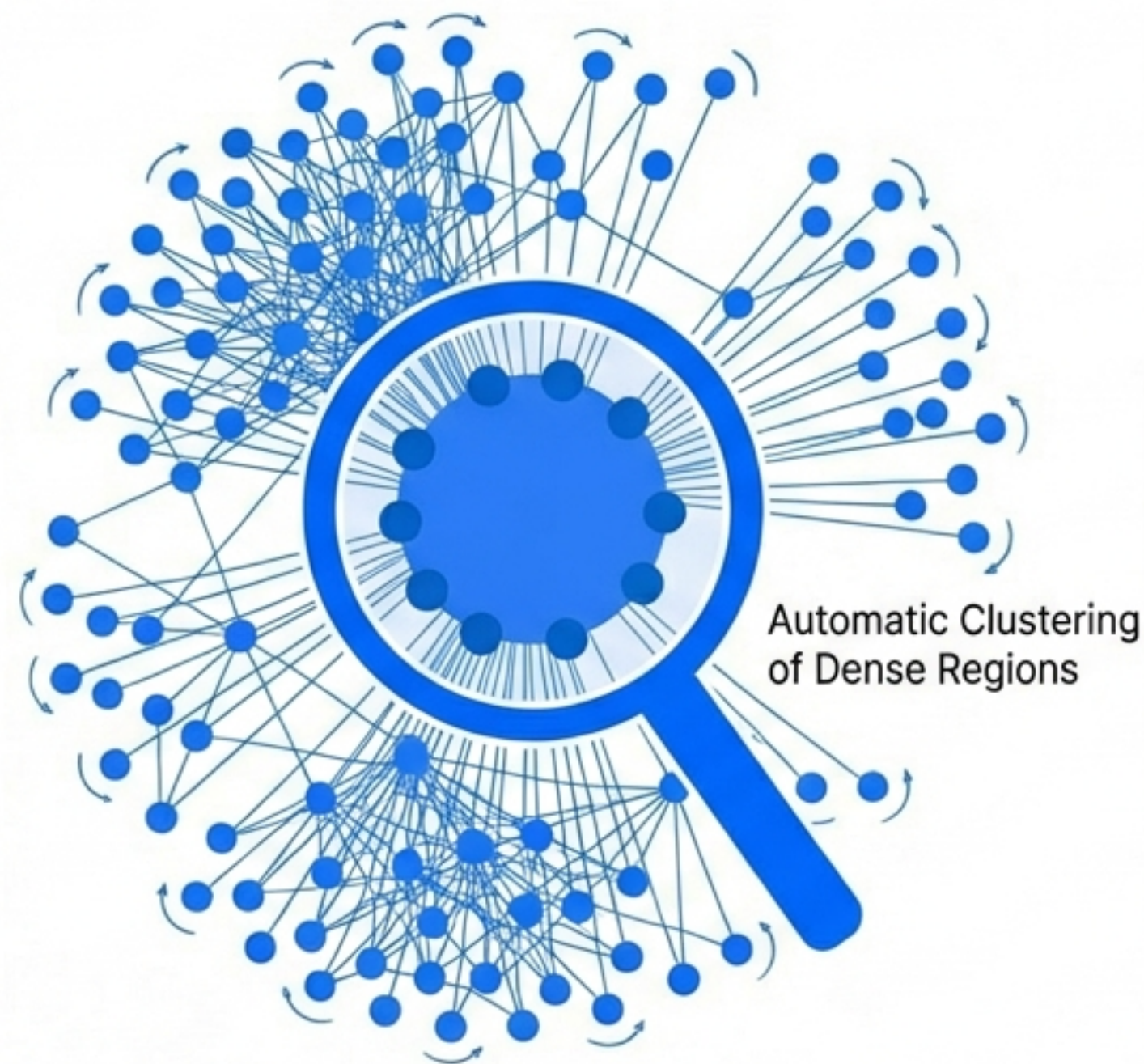
Target Use Case: Large graphs, mobile users, and tactile interaction.

Key Advantages:

- Performance with large datasets
- Physics Engine for natural stabilization
- Touch support & Clustering

Implementation (Task-20):

- Load Vis.js Network CDN
- New Component: 'graph-viewer-visjs.js'
- Layouts: Hierarchical, Physics-based, Circular



Mode 3: Cytoscape.js (The Analyst)

Target Use Case:

Complex network analysis and scientific data representation.

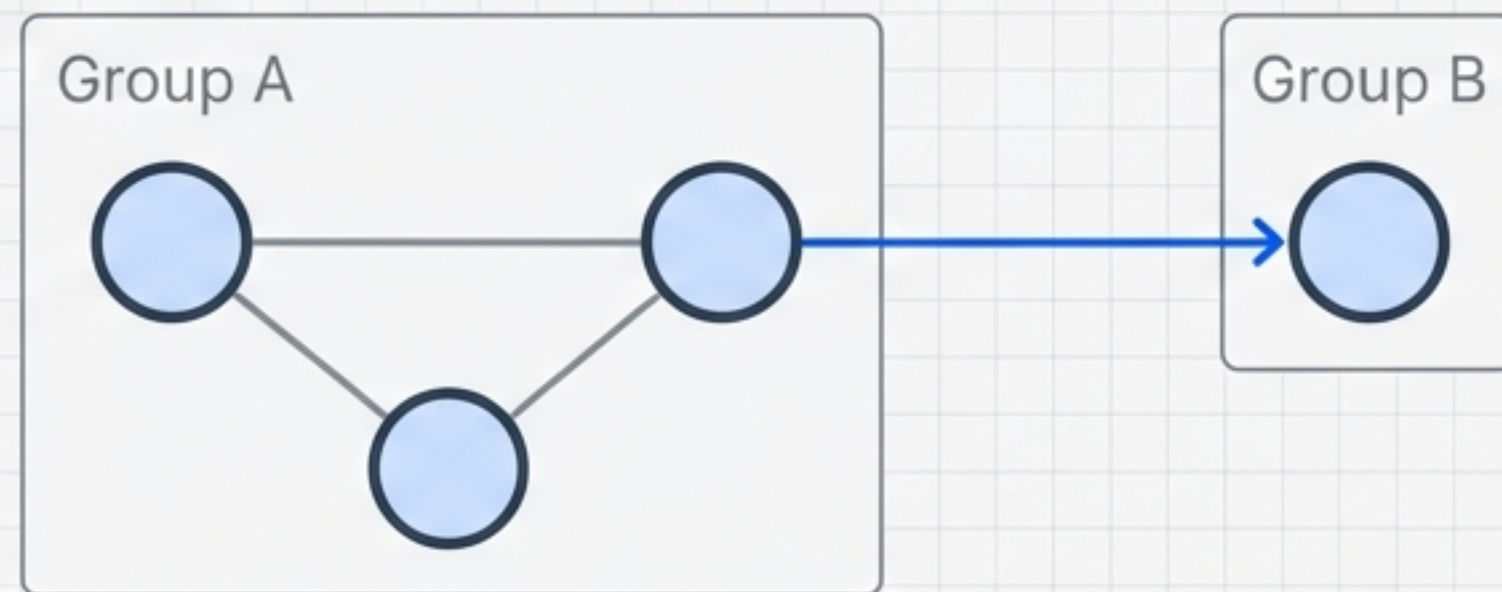
Key Advantages:

- Industry Standard (Bio/Network analysis)
- Powerful Algorithms (Shortest path, Centrality)
- Compound Nodes (Nested groups)

Implementation (Task-21):

- Load Cytoscape.js CDN
- New Component: 'graph-viewer-cytoscape.js'
- Layouts: Cola, Dagre, Breadthfirst

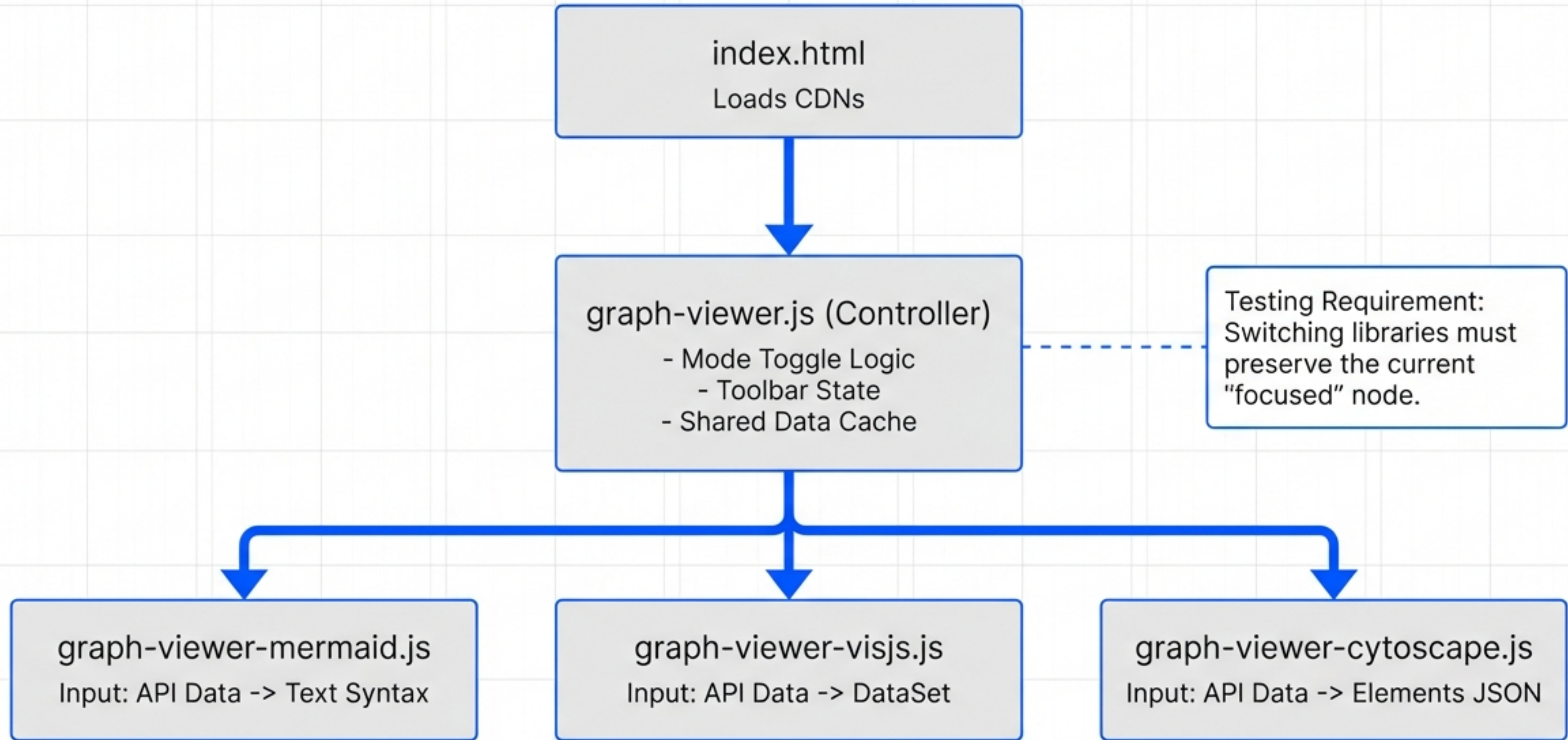
Compound Nodes



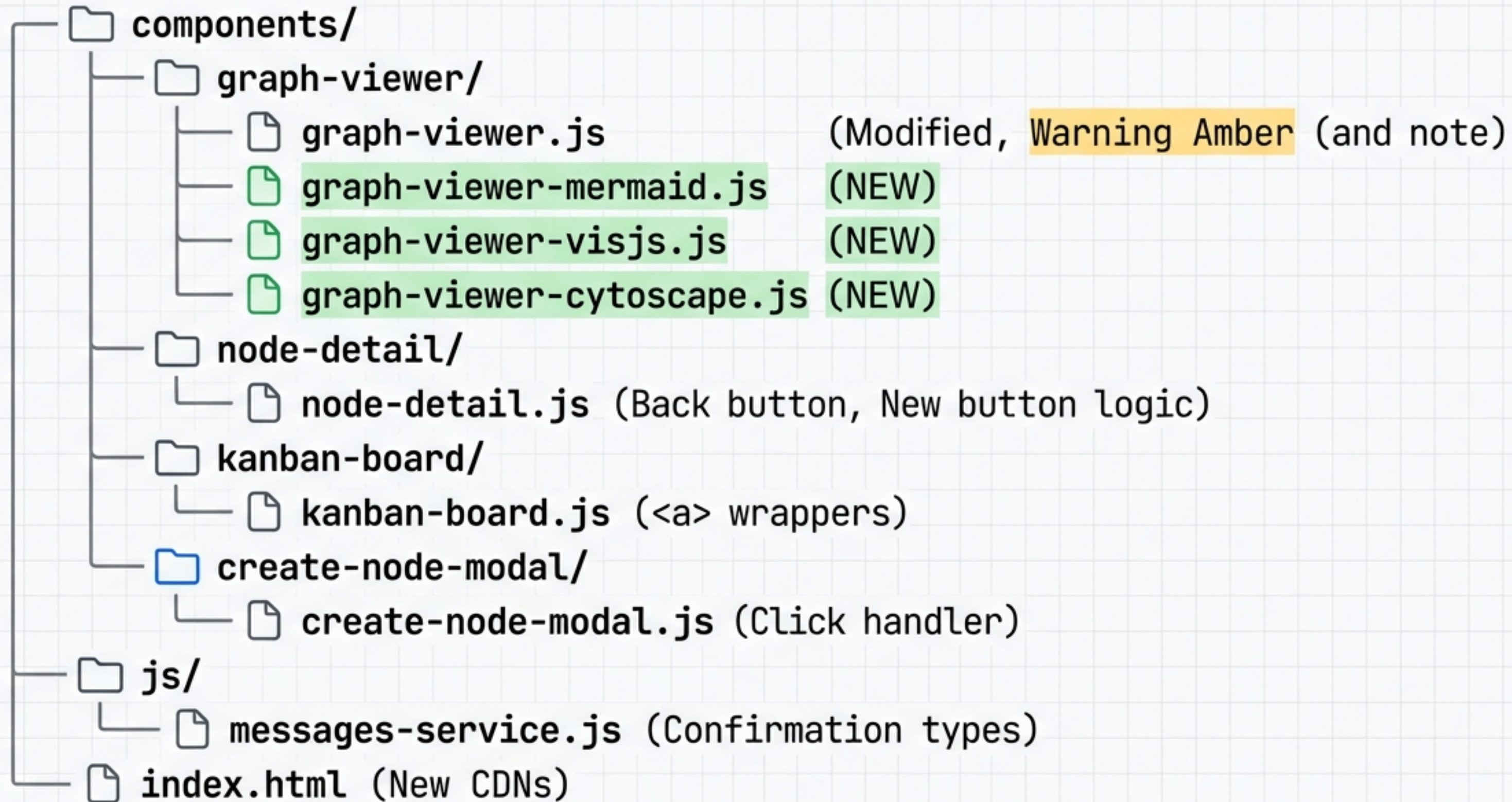
Shortest Path Calculation



Graph Implementation Architecture



Technical Impact & File Structure



Quality Assurance: Critical Checkpoints

UX Validation

- ✓ Modal does NOT close on outside click with data.
- ✓ 'Back' button respects navigation history (Kanban vs List).
- ✓ Cmd+Click on Kanban card opens new tab.
- ✓ Delete action triggers inline panel confirmation.

Graph Validation

- ✓ Toggle switches renderers without page reload.
- ✓ Mermaid export copies valid Markdown syntax.
- ✓ Vis.js physics stabilise smoothly.
- ✓ Cytoscape layouts (Cola/Dagre) render hierarchy correctly.

Risk Assessment & Mitigation Matrix

Medium Risk / High Impact	Risk: Drag-and-drop breaking due to new <a> wrappers in · Kanban.	Mitigation: Strict pointer-events testing; ensure drag starts on 'mousedown'.
Medium Risk / Medium Impact	Risk: Library CSS conflicts / Leaks. Risk: Variable large graph performance.	Mitigation: Namespace all custom styles; use scoped containers. Mitigation: Document performance (Vis.js preferred for scale).
Low Risk / Medium Impact	Risk: CDN availability.	Mitigation: Use reliable sources (jsdelivr, unpkg) with fallback.

Approval & Next Steps

Action Required

Backend Team review of implementation plan and API requirements (specifically regarding data shapes for graph mappers).

Execution Timeline

1. Phase 1: UX Critical Fixes (Bugs 7, 8).
2. Phase 2: Workflow Enhancements (Tasks 18, 23, 25).
3. Phase 3: Graph Library Integration (Tasks 19-21).

Goal: Deployment of v0.1.5 to staging by end of sprint.